

ESP32 5 — Periodic Sensor Data with HTTP POST

Tutorial Group: ESP32

Goal

Starting with *ESP32 4 — HTTP POST and a local Python server*, modify the ESP32 so that it reads the variable resistor and sends its ADC value to the Python server every 10 seconds. This tutorial combines analog input, Wi-Fi, HTTP POST, and `millis()`.

The Python server from *ESP32 4 — HTTP POST and a local Python server* is reused. Do not create a second server.

What You Need

- ESP32-WROOM-32
- USB cable
- Computer with the Arduino IDE and Python 3
- Wi-Fi network
- Variable resistor with the wiper connected to ESP32 pin 34, and the top connected to 3.30v and the bottom connected to GND (0.00v).

Part 1 — Read the Variable Resistor

Use the same variable-resistor circuit from the previous ESP32 tutorials and connect it to pin 34 of the ESP32. Define the analog input:

```
const int analogPin = 34;
```

Read the ADC:

```
int adcValue = analogRead(analogPin);
```

Part 2 — Send the ADC Value

Convert the value to a String so it can be used as the POST body:

```
String postData = String(adcValue);
```

Then send it with:

```
http.POST(postData);
```

Part 3 — Schedule the POST with `millis()`

Create these global variables:

```
unsigned long lastPostTime = 0;  
const unsigned long postInterval = 10000;
```

Inside `loop()`, check whether 10 seconds have elapsed:

```

if ((millis() - lastPostTime) >= postInterval)
{
    lastPostTime = millis();

    // Read and POST the ADC value
}

```

Do not use `delay(10000)`. Because `millis()` is used, the processor can continue executing `loop()` while it waits for the next scheduled POST.

What Happens When the Timer Expires?

The `loop()` function runs repeatedly. Each time through the loop, the program checks whether 10,000 milliseconds have elapsed since the last POST. At startup, `lastPostTime` is 0, so the first POST occurs approximately 10 seconds after the ESP32 starts running the program.

When the condition becomes true, `lastPostTime = millis()` records the current time. The ESP32 then reads the analog input, sends the ADC value to the Python server, prints the HTTP response, and returns to `loop()`. The next POST is scheduled after another 10,000 milliseconds have elapsed.

The if statement does not stop the processor for 10 seconds. When the interval has not elapsed, the condition is false and `loop()` immediately continues. The ESP32 repeatedly checks the elapsed time until the condition becomes true again. This is why `millis()` can schedule a recurring operation without using `delay(10000)`.

Part 4 — Complete Program

```

#include <WiFi.h>
#include <HttpClient.h>

const char* ssid = "YourNetworkName";
const char* password = "YourPassword";
const char* serverURL = "http://192.168.1.50:5000/";

const int analogPin = 34;

unsigned long lastPostTime = 0;
const unsigned long postInterval = 10000;

void setup()
{
    Serial.begin(115200);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED)
    {
        Serial.print(".");
        delay(500);
    }

    Serial.println(" Wi-Fi connected");
}

void loop()
{
    if ((millis() - lastPostTime) >= postInterval)
    {

```

```

    lastPostTime = millis();

    int adcValue = analogRead(analogPin);

    HTTPClient http;
    http.begin(serverURL);

    int responseCode = http.POST(String(adcValue));

    Serial.print("ADC value: ");
    Serial.println(adcValue);

    Serial.print("HTTP Response: ");
    Serial.println(responseCode);

    http.end();
  }
}

```

Part 5 — Observe the Results

Leave the Python server running. You should see values such as:

```

POST received:
1837

POST received:
2011

POST received:
2476

```

Turn the variable resistor and observe the values changing.

Part 6 — Experiment

- Change `postInterval` to 5000. Does the POST occur approximately every 5 seconds?
- Change `postInterval` to 30000. What changes?
- Print `millis()` each time a POST occurs. Are the intervals exactly 10,000 milliseconds? Why might they differ slightly?
- Calculate the voltage from the ADC value and print it locally without changing the POST data.

What You Should Understand

- How to read an analog input on the ESP32.
- How to send an ADC value in an HTTP POST.
- How to use `millis()` to schedule a periodic operation without blocking.
- Why the recurring operation belongs in `loop()`.
- How the Python server receives the periodic values.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I can read the variable resistor.
- ☐ I can send the ADC value in an HTTP POST.

- ☐ I can use `millis()` to schedule a periodic operation.
- ☐ I understand why the recurring operation belongs in `loop()`.
- ☐ I can observe the received values on the Python server.